

110.3. Qui passe devant ? Les trois couches de l'écran

Un héros traverse un mur, puis le panneau : il passe devant les deux. L'écran est fait de trois couches, toujours dans le même ordre.

1. Ce qu'il faut comprendre

Une question que tout jeu se pose : quand deux choses sont au même endroit de l'écran, laquelle voit-on ? Le héros devant le mur, ou le mur devant le héros ?

La réponse de la Game Boy : trois couches, empilées comme des feuilles transparentes. Tout au fond, le **décor** (ce que posent `poser()` et `texte()`). Par-dessus, le **panneau** (`panneau()`, `textePanneau()`, `poserPanneau()`, le chapitre 9). Tout devant, les **lutins** (`sprite()`).

Ce qui est nouveau ici : rien à apprendre, tout à regarder. Le programme pose les trois éléments : un mur de briques au milieu (le décor), une colonne grise à droite avec « SCORE » (le panneau, placé au pixel 120), et un héros qui traverse l'écran tout seul (un lutin). Le héros passe **devant le mur**, puis **devant le panneau**.

L'ordre des lignes ne compte pas. On pourrait poser le mur APRÈS le premier `sprite()` : il resterait derrière. Ce n'est pas « le dernier dessiné gagne », comme sur une feuille de papier : chaque chose est rangée dans SA couche, et la console empile toujours les couches dans le même ordre.

Le panneau, au pixel 120 : `panneau(120, 0)` le pose à partir du pixel 120 en largeur (la colonne 15) et du pixel 0 en hauteur. Il couvre alors tout ce qui est à droite et en dessous : les colonnes 15 à 19, du haut en bas. Le décor, sous lui, ne se voit plus.

À toi : déplace le mur (colonnes 3 et 4 par exemple) et regarde : le héros passe toujours devant. Les leçons suivantes montrent comment changer cet ordre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poserPanneau>` → [110.1. La fonction poserPanneau\(\) — une tuile dans le panneau](#)
- `#include <textePanneau>` → [60.2. La fonction textePanneau\(\) — écrire dans le panneau](#)
- `#include <panneau>` → [60.1. La fonction panneau\(\) — montrer le panneau](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **730 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage – la mise en route de la console
main()	188 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), poserPanneau() – trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte(), textePanneau() – écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	76 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	256 o	← CopierTuiles – 13 lettres sur 43, et 3 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins
"QUI EST DEVANT"	14 o	← texte() – un texte gravé
"SCORE"	5 o	← textePanneau() – un texte gravé

Lettres gravées : ACDEINOQRSTUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

poser()	x 2	24 o sur place, dans main() + AdresseCase (10 o)
sprite()	x 1	23 o sur place, dans main()
panneau()	x 1	15 o sur place, dans main()
poserPanneau()	x 1	13 o sur place, dans main() + AdresseCase (10 o)
texte()	x 1	11 o sur place, dans main() + EcrireTexte (10 o)
textePanneau()	x 1	11 o sur place, dans main() + EcrireTexte (10 o)
image()	x 1	6 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un héros traverse l'écran tout seul, de gauche à droite.
// Il passe sur un MUR (le décor), puis sur une colonne grise à droite (le PANNEAU).
// Regarde bien : il passe DEVANT les deux.
//
// Ce qui est nouveau : rien à écrire, tout à REGARDER.
// L'écran de la Game Boy est fait de TROIS COUCHES, toujours dans le même ordre :
//
//      toi, tu regardes d'ici
//      |
//      v
//  3. les LUTINS    sprite()                <- tout devant
//  2. le PANNEAU    panneau(), poserPanneau(), textePanneau()
//  1. le DÉCOR      poser(), texte()         <- tout au fond
//
// L'ordre des lignes du programme ne compte pas : chaque chose va dans SA
// couche, et la console empile toujours les couches dans cet ordre-là.
```

```
#include <Tuile>           // un dessin de 8 × 8 pixels
#include <poser>            // pose une tuile sur une case du fond (le DÉCOR)
#include <texte>            // écrit un texte à l'écran (le DÉCOR aussi)
#include <poserPanneau>     // pose une tuile dans le PANNEAU
#include <textePanneau>     // écrit un texte dans le PANNEAU
#include <panneau>          // montre le panneau, à une place choisie
#include <sprite>           // place un LUTIN de 8 × 8 au pixel près
```

2. UN DESSIN — BRIQUE

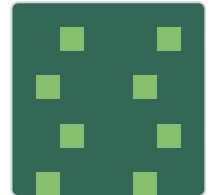
```
Tuile BRIQUE = {                                // Le mur : des joints en 3 (sombres), des briques en 1
(claire).
  "33333333",
  "11131111",
  "11131111",
  "33333333",
  "13111113",
  "13111113",
  "33333333",
  "11131111",
};
```



à l'écran

3. UN DESSIN — GRIS

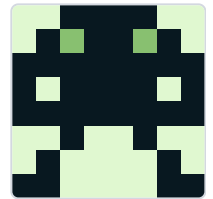
```
Tuile GRIS = {                                  // Le fond du panneau : du gris (2), piqué de clair (1).
  "22222222",
  "22122212",
  "22222222",
  "21222122",
  "22222222",
  "22122212",
  "22222222",
  "21222122",
};
```



à l'écran

4. UN DESSIN — HEROS

```
Tuile HEROS = {                                // Le héros. Les points (.) sont TRANSPARENTS chez un lutin.
  "..####..",
  ".#-##-#.",
  "#####",
  "#.####.#",
  "#####",
  "..#..#..",
  ".#...#.",
  "##...##",
};
```



à l'écran

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.

    // ---- 1. LE DÉCOR (tout au fond) : un mur, colonnes 9 et 10, lignes 7 à 11.
    for (uint8_t l = 7; l < 12; l++) {      // Pour chaque ligne l, de 7 à 11 :
        poser(9, l, BRIQUE);                // une brique en colonne 9,
        poser(10, l, BRIQUE);              // une autre en colonne 10.
    }
    texte(1, 1, "QUI EST DEVANT");          // Le texte aussi est dans le décor.

    // ---- 2. LE PANNEAU (au milieu) : rempli de gris, avec SCORE en haut.
    // On le remplit d'abord : 5 colonnes (0 à 4) sur 18 lignes (0 à 17).
    for (uint8_t l = 0; l < 18; l++) {
        for (uint8_t c = 0; c < 5; c++) {
            poserPanneau(c, l, GRIS);        // Les colonnes du PANNEAU, pas celles de l'écran.
        }
    }
    textePanneau(0, 1, "SCORE");             // Colonne 0 du panneau, ligne 1.
    panneau(120, 0);                        // On le montre : à partir du pixel 120 (la colonne 15)
                                           // et du pixel 0 en hauteur. Il couvre tout le côté droit.

    // ---- 3. LE LUTIN (tout devant) : le héros, qui avance tout seul.
    uint8_t x = 0;                          // Sa place, en pixels, de gauche à droite.

    while (true) {                          // La boucle du jeu :
        image();

        if (images() % 2 == 0) x++;          // Une image sur 2 : un pixel de plus vers la droite.
        if (x > 152) x = 0;                 // Sorti à droite ? Il repart de la gauche.

        sprite(0, x, 72, HEROS);            // Ligne de pixels 72 : à la hauteur du mur (ligne 9).
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un mur de briques au milieu, une colonne grise « SCORE » à droite. Un petit héros traverse l'écran : il passe devant le mur, puis devant le panneau.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le mur est dans le décor, le panneau à droite (1, 2)
- ✓ le héros passe DEVANT le mur (nuance 3)
- ✓ le héros passe DEVANT le panneau (nuance 3)

Ensuite : **110.4. Devant, derrière : le tuyau, DERRIERE** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++